



Open Soft Competition | 2026

Title: Project Synthetic-Bull: Full-Stack Exchange Simulator & Algorithmic Trading Terminal

Background & Objective:

At NextBull, before we deploy any new trading interface or execution algorithm into live markets, we test them in highly controlled, simulated environments. We are challenging the tech innovators at IIT Kharagpur to build **Project Synthetic-Bull**, a completely self-contained, real-time simulated cryptocurrency/stock exchange with an integrated web trading terminal and automated quantitative trading bots.

This project sits at the intersection of high-throughput backend engineering, real-time frontend visualization, and algorithmic trading. **No external datasets will be provided.** Your system must generate its own synthetic market activity, maintain a live order book, and allow both human and AI users to trade against it.

Technical Requirements & Specifications:

Module 1: The Core Matching Engine & Market Simulator (Backend / Systems)

- **The Matching Engine:** Build an in-memory Limit Order Book (LOB) that supports three strict message types: Limit Order, Market Order, and Cancel Order. It must execute trades using strict Price-Time priority.
- **The Synthetic Market Generator:** Since no external data is provided, you must build a background process that acts as the "Market." This process will continuously generate a baseline asset price using a Geometric Brownian Motion (GBM) model:
$$S_t = S_0 \exp\left(\left(\mu - \frac{\sigma^2}{2}\right)t + \sigma W_t\right)$$

Based on this moving baseline price, the generator must continuously submit a flurry of randomized Limit Bids and Asks to populate the order book and simulate real market liquidity.

Module 2: The NextBull Web Terminal (Frontend / SDE)

- **Real-Time Data Streaming:** The backend must broadcast order book updates and trade executions to the frontend using WebSockets.
- **The Dashboard:** Build a modern, NextBull-styled web interface featuring:
 - A live, auto-updating Candlestick chart (1-second or 5-second intervals).
 - A visual representation of the Limit Order Book (Bid/Ask depth).
 - A control panel for a "Human User" to manually submit Market and Limit orders.
 - A real-time Portfolio widget showing the user's current cash balance, asset holdings, and live P&L (Profit and Loss).

Module 3: The Quantitative Trading Bots (Quant / ML)

Note : This part is optional for the teams we will use this to eliminate ties

Design and integrate two distinct, automated trading bots that connect to your exchange via WebSockets or local API and trade with simulated capital:

- **The Market Maker Bot:** A bot that constantly provides liquidity by placing Limit Bids slightly below the mid-price and Limit Asks slightly above the mid-price, attempting to profit from the spread while managing its inventory risk.
- **The Alpha Bot:** A directional trading bot. This bot must ingest the live data stream and use a statistical or ML approach (e.g., Moving Average Crossover, RSI, or a lightweight predictive model) to execute Market Orders when it detects a trend.

Constraints & System Rules:

1. **Zero External Dependencies for Data:** The system must not rely on live APIs (like Binance or Yahoo Finance) or external CSV files. The market simulation must be generated 100% locally by your code upon startup.
2. **Latency & Scale:** Your synthetic market generator should push at least 50-100 order messages per second to test the capability of your WebSocket stream and React/Frontend rendering.
3. **Capital Rules:** All users and bots start with a simulated \$100,000. Short selling is permitted.

Submission Instructions & Deliverables:

1. **Source Code:** Fully containerized. A single docker-compose up command must launch the backend engine, the market generator, the trading bots(optional), and the frontend terminal simultaneously.
2. **Architecture & Quant Report:** A document (max 10 pages) containing:
 - System architecture diagram mapping the data flow from the Matching Engine to the Frontend.
 - The mathematical logic/parameters used for your Synthetic Market Generator and Trading Bots.(**This point is optional**)
3. **Presentation explaining the solution (max 15 slides): 10% weightage**

Evaluation Criteria (100 Points Total):

- **Backend & Architecture (20%):** Accuracy of the matching engine, WebSocket stability, and ability to handle high message throughput without crashing.
- **Frontend & UX (50%):** Responsiveness of the web terminal, quality of the charting, and intuitive UI/UX design.
- **Quant & Bot Logic (5% Bonus):** The realism of the synthetic market data generator and the logical soundness/profitability of the automated trading bots.
- **Code Quality & Deployment (20%):** Clean, modular code, proper Dockerization, and seamless 1-click execution.